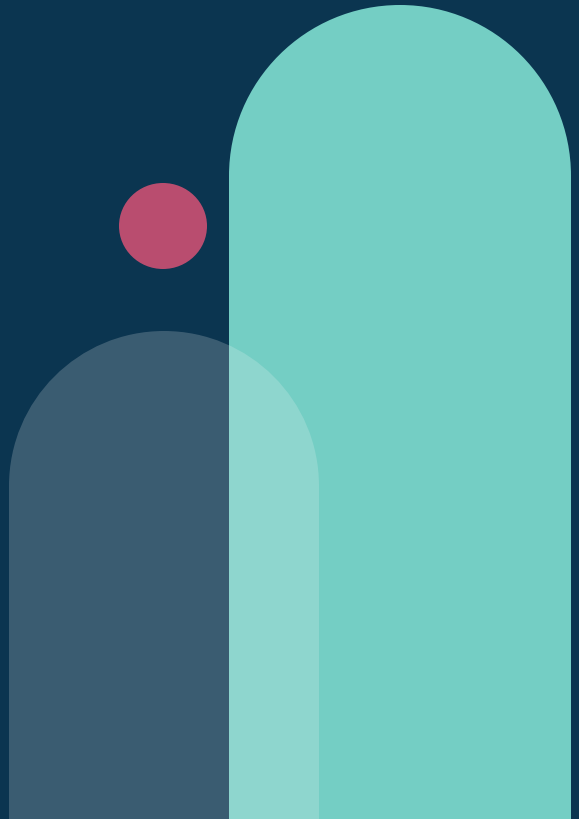
Python for Generative AI

- Basics of Python
- Intro to Jupyter Notebook

01

Basics of Python



Why Python for Gen AI?



Extensive Libraries



Readability and Ease of Use



Large and Supportive Community



Platform Independence

Installation

Install Python

- https://www.python.org/downloads/

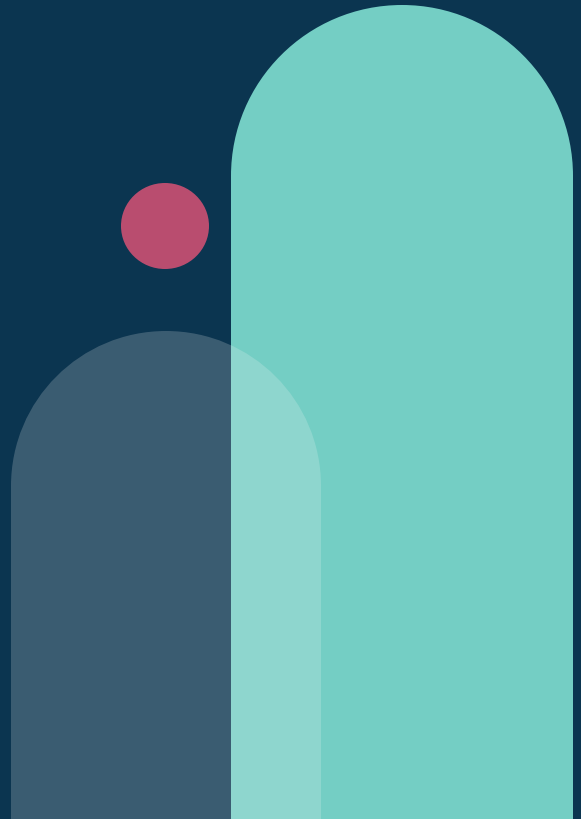
Install VSCode

- https://code.visualstudio.com/download

01

Basics of Python

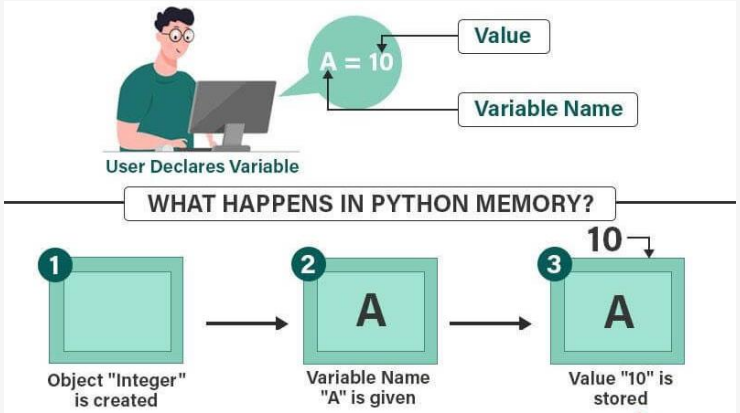
- Variables
- Datatypes
- Operators
- Control Flow
- Functions



Python Variables

What is a Variable?

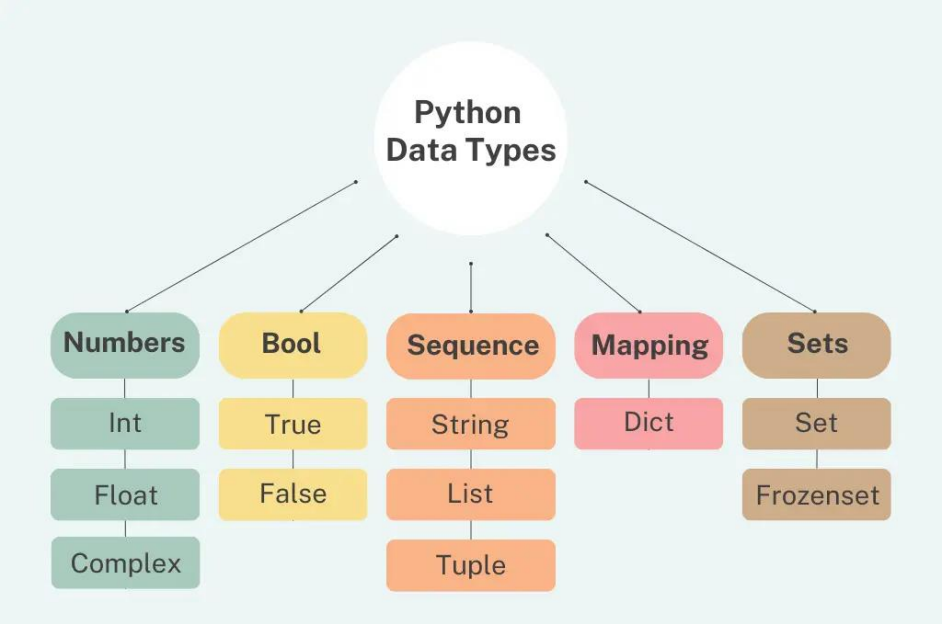
- A variable is a name that refers to a value stored in memory.
- Variables are used to store data that can be manipulated and retrieved later.



Rules for Naming Variables

- Must start with a letter or an underscore (_).
- Cannot start with a number.
- Can contain letters, numbers, and underscores.
- Are case-sensitive (myVar, MyVar, and myvar are different).

Python Datatype



Python Datatype

Name	Type	Description
Integers	int	Whole numbers, such as: **3 300 200**
Floating point	float	Numbers with a decimal point: **2.3 4.6 100.0**
Strings	str	Ordered sequence of characters: **"hello" 'Sammy' "2000" "楽しい"**
Lists	list	Ordered sequence of objects: **[10,"hello",200.3]**
Dictionaries	dict	Unordered Key:Value pairs: **{"mykey" : "value" , "name" : "Frankie"}**
Tuples	tup	Ordered immutable sequence of objects: **(10,"hello",200.3)**
Sets	set	Unordered collection of unique objects: **{"a","b"}**
Booleans	bool	Logical value indicating **True** or **False**

Python Variables with Data Type

Example Code

```
# Assigning values to variables
age = 25
name = "Alice"
is_student = True

# Checking the type of a variable
print(type(age))       # Output: <class 'int'>
print(type(name))      # Output: <class 'str'>
print(type(is_student)) # Output: <class 'bool'>
```

Best Practices

- Use meaningful variable names (e.g., student_age instead of x).
- Stick to lowercase with underscores for multi-word variables (e.g., total_price).
- Avoid using Python reserved words as variable names (e.g., if, for, while).

Understanding Lists and Dictionaries

Lists:

- **Definition**: Ordered, mutable collections of items.
- **Creation**: my_list = [1, 2, 3, 'hello']
- **Accessing Items**: Use indices to access items. my_list[0] returns 1.
- **Operations**: Add (append(), insert()), remove (remove(), pop()), and modify items.
- **Use Case**: Perfect for keeping a collection of items where order matters and might need modification.

Dictionaries:

- **Definition**: Collections of key-value pairs.
- **Creation**: my_dict = {'name': 'John', 'age': 30}
- **Accessing Items**: Use keys to access values. my_dict['name'] returns 'John'.
- **Operations**: Add or modify items (my_dict['age'] = 31), delete items (del my_dict['age’]).
- **Use Case**: Ideal for data that needs to be retrievable via unique keys.

Exploring Tuples and Sets

Tuples:

- **Definition**: Ordered, immutable collections of items.
- **Creation**: my_tuple = (1, 2, 3, 'hello')
- **Accessing Items**: Use indices. my_tuple[0] returns 1.
- **Use Case**: Use when you need an ordered collection that should not be changed.

Sets:

- **Definition**: Unordered collections of unique elements.
- **Creation**: my_set = {1, 2, 3, 'hello'}
- **Operations**: Add (add()), remove (remove(), discard()), check membership (in).
- **Use Case**: Great for membership testing, removing duplicates from a sequence, and performing mathematical operations like unions, intersections, and differences.

Operators

Operators	Type
+, -, *, /, %	Arithmetic operator
<, <=, >, >=, ==, !=	Relational operator
AND, OR, NOT	Logical operator
&, |, <<, >>, -, ^	Bitwise operator
=, +=, -=, *=, %=	Assignment operator

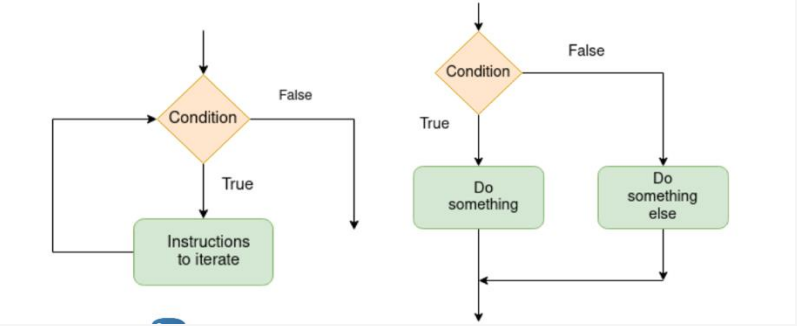
Control Flow

What is Control Flow?

- Control flow refers to the order in which the code is executed in a program.
- Python uses control flow statements to make decisions and repeat actions.

Key Control Flow Statements:

- **Conditional Statements**: if, elif, else
- **Loops**: for, while
- **Break and Continue**: Modify loop behavior



Control Flow

Conditional Example

```
age = 18
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Conditional Example

```
for i in range(5):
    print(i)  # Outputs: 0, 1, 2, 3, 4
```

Break and Continue

- Break: Exits the loop prematurely.
- Continue: Skips the rest of the code inside the loop for the current iteration and moves to the next iteration.

Control Flow

Example

```
# Loop through a range of numbers
for num in range(1, 11):

    # If the number is 5, break the loop
    if num == 5:
        print("Breaking the loop at number", num)
        break

    # If the number is even, skip to the next iteration
    elif num % 2 == 0:
        print(f"{num} is even, continuing to next iteration.")
        continue

    # If the number is odd, print it
    else:
        print(f"{num} is odd.")
```

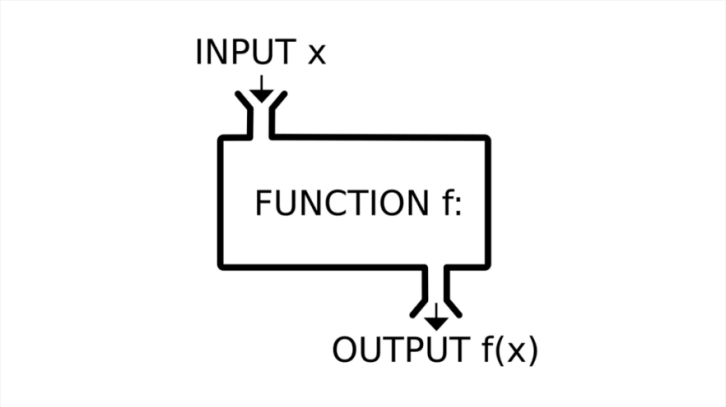
Output

```
1 is odd.
2 is even, continuing to next iteration.
3 is odd.
4 is even, continuing to next iteration.
Breaking the loop at number 5
```

Functions

What is a Function?

- A function is a block of organized, reusable code that performs a single action.
- Functions help break programs into smaller, modular chunks.



Defining a Function:

- Use the def keyword followed by the function name and parentheses.
- Example:

```
def greet(name):
    print(f"Hello, {name}!")
```

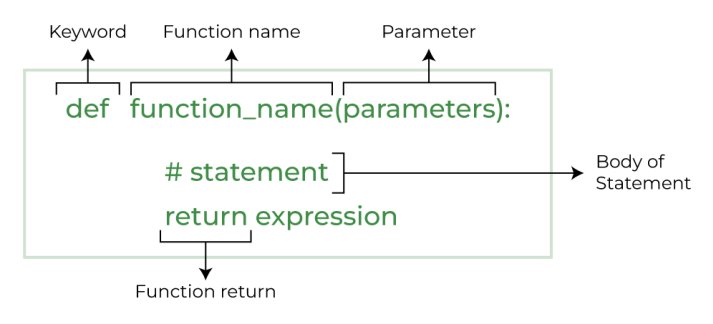
Functions

Parameters and Arguments:

- **Parameters**: Variables listed inside the parentheses in the function definition.
- **Arguments**: Values passed to the function when it is called.

Return Values:

- Functions can return a value using the return statement.
- The returned value can be stored in a variable or used directly.



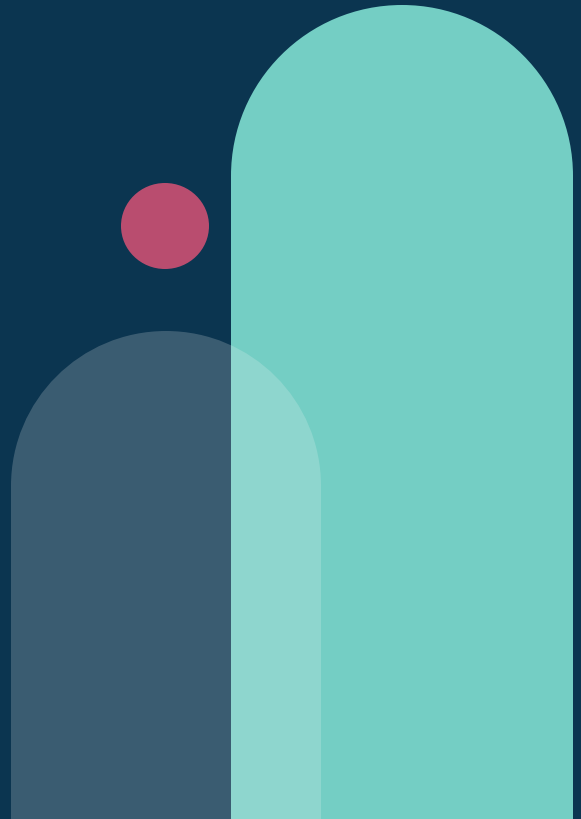

```
def add(a, b):
    return a + b

result = add(3, 4)  # Arguments passed to the function
print(result)       # Outputs: 7
```

01

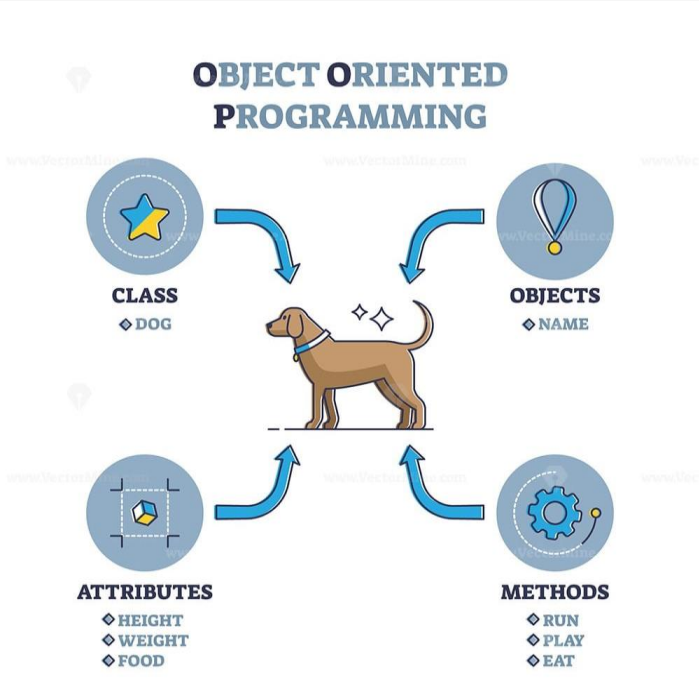
Basic of Python

- OOPS
- Classes and Objects
- Inheritance and Polymorphism
- Encapsulation and Abstraction



Introduction to OOP in Python

- **What is OOP?**
  - A programming paradigm based on the concept of "objects," which can contain data and code.
  - Objects represent real-world entities and provide a way to structure software in a more natural and modular way.
- **Key Concepts**:
  - **Class**: A blueprint for creating objects.
  - **Object**: An instance of a class.
  - **Attributes**: Variables that belong to an object.
  - **Methods**: Functions that belong to an object.



Classes and Objects

Defining a Class

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
```

Creating an Object

```
my_car = Car("Toyota", "Corolla")
```

Accessing Attributes

```
print(my_car.brand)  # Outputs: Toyota
print(my_car.model)  # Outputs: Corolla
```

Inheritance and Polymorphism

Inheritance

- Allows a class to inherit attributes and methods from another class.

```
class Vehicle:
    def __init__(self, name):
        self.name = name

    def drive(self):
        print(f"{self.name} is driving.")

class Car(Vehicle):
    def honk(self):
        print("Beep! Beep!")
```

Polymorphism

- Allows different classes to be treated as instances of the same class through inheritance.

```
my_vehicle = Car("Sedan")
my_vehicle.drive()  # Outputs: Sedan is driving.
my_vehicle.honk()   # Outputs: Beep! Beep!
```

Encapsulation and Abstraction

Encapsulation

- Bundling of data (attributes) and methods (functions) that operate on the data into a single unit or class.
- Controls access to the internal state of an object using private attributes/methods.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance  # Private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Abstraction

- Hiding the complex implementation details and showing only the essential features.
- Allows users to interact with objects at a higher level.

```
account = BankAccount(100)
account.deposit(50)
print(account.get_balance())  # Outputs: 150
```

02

Jupyter Notebook

What is Jupyter Notebook?

Overview:

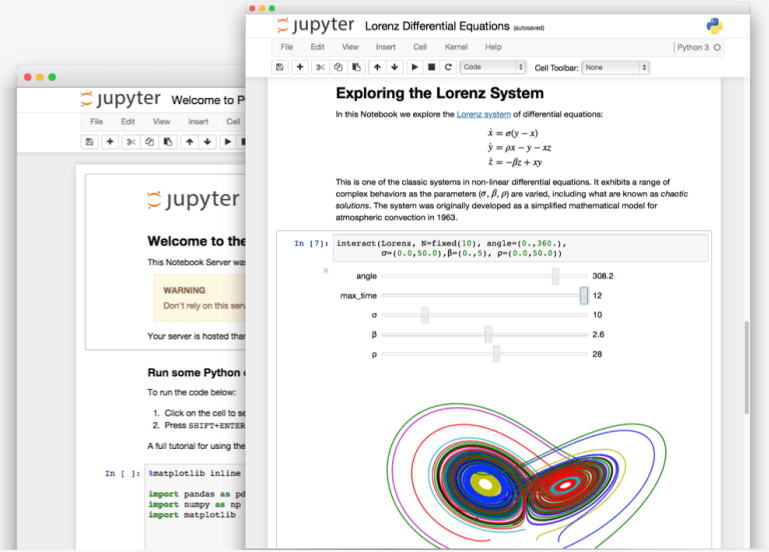
- Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.

Key Features:

- Supports over 40 programming languages, including Python, R, and Julia.
- Ideal for data cleaning, transformation, numerical simulation, statistical modeling, and machine learning.

Use Cases:

- Data analysis
- Prototyping
- Teaching and learning
- Collaboration



Conda Installation

What is Conda?

- Conda is a package, dependency, and environment management tool that works across platforms (Windows, macOS, Linux).
- Anaconda includes Conda, Python, and over 1,500 data science packages.

What is Miniconda?

- Miniconda is a lightweight version of Anaconda that includes only Conda, Python, and a few basic packages.

Install Miniconda

- https://docs.anaconda.com/miniconda/miniconda-install/

Conda Environment Creation

Why Use Environments?

- Isolate project dependencies to avoid conflicts between different projects.

Creating a New Environment:

- `conda create -n myenv python=3.10`
  - `myenv` is the name of the environment.
  - `python=3.10` specifies the Python version.

Activating the Environment:

- `conda activate myenv`

Deactivating the Environment:

- `conda deactivate`

Installing and Launching Notebook

Installing Jupyter Notebook:

- With your Conda environment activated, install Jupyter Notebook using
  - conda install jupyter

Launching Jupyter Notebook:

- jupyter notebook
- This will open the Jupyter Notebook interface in your web browser.

Using Jupyter Notebook:

- Create new notebooks, open existing ones, and manage files within the Jupyter interface.
- Write and execute code cells, markdown cells, and use interactive widgets.

Thanks!

Do you have any questions?

